CLIMATE INTELLIGENCE

Climate Intelligence for Heatwave Monitoring, Prediction and Early Warning

CLIMATE DATA PROCESSING AND EARLY WARNING

From heterogeneous observations to interpretable signals

Student name: _____ Roll number: _____

Class / division: _____ Course: _____ Institution: _____

1 CLIMATE QUESTION AND ANALYTICAL OBJECTIVE

How can heterogeneous coursework weather observations be placed in one local register and summarized into understandable heat-risk cues? Climate Intelligence maps selected CSV fields to a common record, validates values, saves them in SQLite and derives deterministic summaries and labels.

DATASET	INITIAL / MANUAL PATH	ANALYTICAL ROLE
weather_data exp 03.csv	50-row bootstrap	Temperature, humidity, wind and condition
Exp04_IMD_50_Real_Weather_Observations.csv	50-row bootstrap	Temperature, rainfall and source text (not authenticated)
Pune_Climate_50_Observations EXP 6.csv	50-row bootstrap	Time, station, temperature and humidity
Weather data (EXP5).csv	24,070-row manual import	20-field weather file; provider unidentified

The two XLSX companions are retained, but the GUI importer reads CSV. First launch inserts 150 rows only when the SQLite database is empty. The larger file must be imported manually; import does not deduplicate. The database stores selected normalized fields, not the full source row.

KEY MEASURE	IMPLEMENTED MEANING
Heat filter	Temperature >=40°C; a coursework filter, not a public criterion
Alert labels	NORMAL <37°C; WARNING 37..<40°C; CRITICAL >=40°C
Trend estimate	Up to eight recent readings; one average per-reading delta added to latest
Stored values	Celsius, humidity %, wind km/h, rainfall mm; one observations table; indexes on time and station

2 NORMALIZATION AND QUALITY CONTROLS

weather_data.py accepts different headers through aliases, prefers Celsius where available and converts miles per hour when needed. It uses DictReader, maps selected time/station/condition fields, supplies defaults for missing humidity, wind and rainfall, and skips rows without a usable temperature. climate_engine.py then validates required values and numeric ranges before a database write.

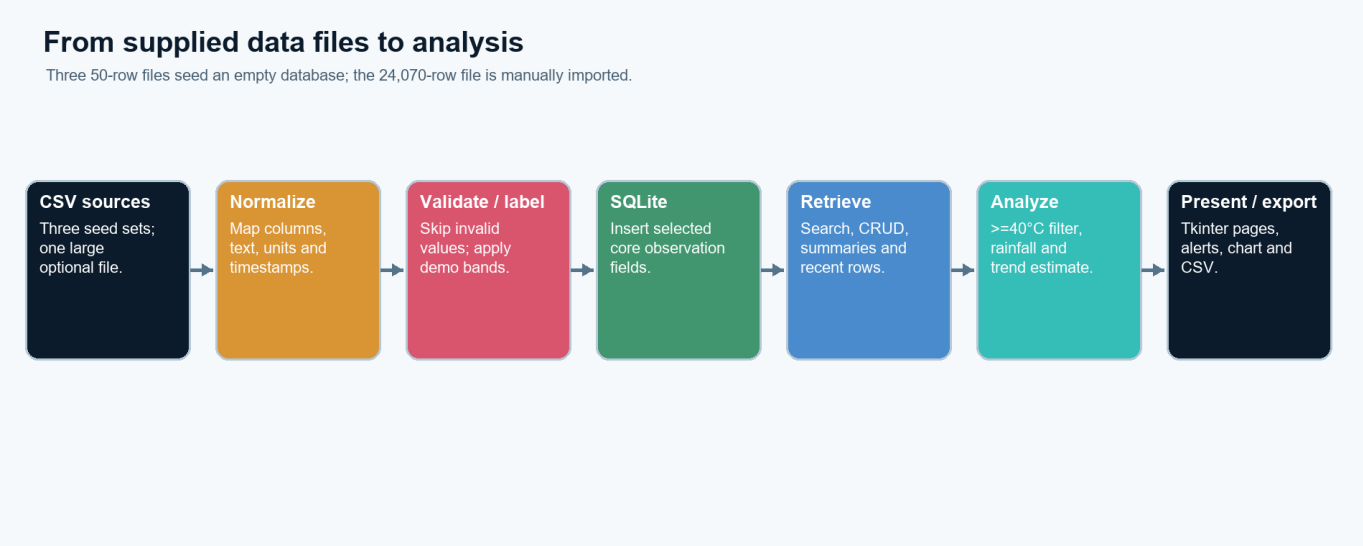


Figure 1. Selected fields move from supplied CSV files through validation into analysis.

CSV row intake · weather_data.py

```
reader = csv.DictReader(handle)
if not reader.fieldnames:
 raise ValueError("This CSV does not contain a header row.")
for line_number, raw in enumerate(reader, start=2):
 row = _row_lookup(raw)
 temp = _pick(row, ["temperature_c", "temperature_celsius", "temperature", "temp_c", "maximum_temperature", "feels_like_celsius"])
 if temp is None:
 continue
```

Validation constrains temperature to −20…60°C and humidity to 0…100%. Original files stay on disk; source identity is not preserved per row. The Exp 04 label claims an IMD source in the supplied cells, but that claim was not independently authenticated.

3 ANALYSIS, WARNING LOGIC AND ESTIMATE

The temperature analyzer creates a list of numeric readings, filters the >=40°C subset and reduces a total used for summary statistics. It also returns mean, extrema, converted Fahrenheit values and the set of stored alert categories. A parallel RainfallAnalyzer provides rainfall totals and wet-reading counts through the same abstract analyze() interface.

Temperature analysis · climate_engine.py

```
temperatures = list(map(lambda r: float(r["temperature"] if isinstance(r, dict) else r.temperature), observations))
hot = list(filter(lambda temp: temp >= 40.0, temperatures))
total = reduce(lambda a, b: a + b, temperatures, 0.0)
square_first = (lambda value: value * value)(temperatures[0]) if temperatures else None
return {"count": len(temperatures), "average": total / len(temperatures) if temperatures else 0.0,
 "minimum": min(temperatures) if temperatures else None,
 "maximum": max(temperatures) if temperatures else None,
 "total": total, "square_first": square_first, "hot_count": len(hot), "hot_values": hot,
 "fahrenheit": list(map(celsius_to_fahrenheit, temperatures)),
 "alert_levels": {classify_alert(temp) for temp in temperatures}}
```

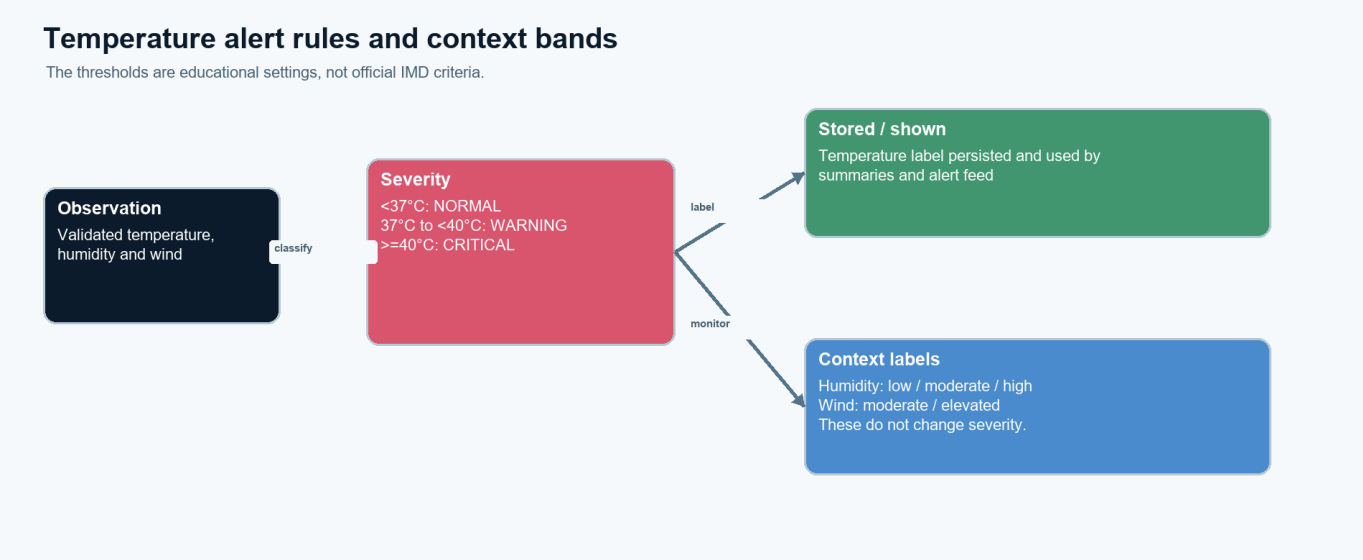


Figure 2. Temperature controls severity; humidity and wind remain descriptive monitor labels.

Temperature classification · alert_engine.py

```
if temp >= CRITICAL_C:
 return "CRITICAL"
if temp >= WARNING_C:
 return "WARNING"
return "NORMAL"
```

The simple prediction screen adds the average first-to-last per-record change from up to eight recent values once to the latest value. Timestamps are not normalized to equal intervals, and no future holdout accuracy is reported. Concurrent checks and simulation are local synthetic workflows, not sensor or API feeds.

4 EXPERIMENT COVERAGE AND CLIMATE USE

Each supplied experiment is represented where supported by the final project. The experiment table distinguishes climate functionality from separate academic support exercises.

EXP. / TOPIC	PYTHON CONCEPT	APPLICATION EVIDENCE
01 · Class creation	Classes, objects, constructor, attributes and encapsulation.	Observation class stores measurements; separate marks classes are not climate analytics.
02 · Constructors and lists	Default/parameterized construction, list accumulation and CRUD.	CSV dictionaries are accumulated then validated into the common observation table.
03 · String handling and inheritance	Whitespace/string methods, base and derived classes, super().	Header/value text is normalized; inherited imports share base observation behavior.
04 · Abstract classes and polymorphism	ABC, abstract method, overriding and a shared method contract.	A common analyze() contract allows temperature and rainfall summaries to be called consistently.
05 · Exceptions, modules and multithreading	Exceptions, module separation, background/concurrent work.	Bad rows are skipped or rejected; concurrent checks classify saved records without an external feed.
06 · Collections and functional programming	Lists, mappings, sets, lambda, map, filter and reduce.	map/filter/reduce and a set drive conversion, >=40°C counts, totals and alert-category summaries.

Trend calculation · app.py

```
vals=self.chart_values[-8:]
estimate=vals[-1]
if len(vals)>=2:
 delta=(vals[-1]-vals[0])/max(1,len(vals)-1)
 estimate=vals[-1]+delta
```

Database and simulation boundary

SQLite contains one observations table with 12 columns and indexes on observed_at and station. The 41.6°C Nagpur Heatwave Demo is explicitly saved as synthetic; the continuous simulation generates local random records. Dashboard summaries can therefore mix imported, manual and synthetic observations.

5 RESULTS, LIMITATIONS AND CONCLUSION

VALIDATION	RESULT
Parsing / scale	Each small CSV yields 50 records; the 24,070-row file parser maps core fields.
Database	150-row seed, CRUD and 50-row CSV import/export round trip pass.
Rules / model	36.9, 37.0 and 40.0°C boundary tests pass; both analyzer implementations are exercised.
Monitoring	GUI smoke confirms all pages, simulation persistence and stored 41.6°C CRITICAL example.
Overall	16/16 automated tests passed, 13 Oct 2026; no predictive-accuracy evaluation was run.

Interpretation and conclusion

The app turns mixed inputs into a usable teaching workflow, not a climatological series. Its imports have incomplete provenance, duplicate rows can accumulate, and the supplied spreadsheets are not accepted by the GUI. Alert cutoffs are educational rather than official; the map is illustrative. The per-record trend is neither a time-calibrated forecast nor machine learning. With these qualifications, normalization, filtering, aggregation and persisted alert labels make the data path traceable and demonstrable.

Selected references

[1] Python Software Foundation. Python 3 Library Reference: tkinter, csv, sqlite3 and concurrent.futures. [2] SQLite Consortium, SQL Language Reference. [3] Supplied Python experiments 01–06 and project source files, preserved in the project workspace.